MRA 检测模型代码分析报告

Codex

2026 年 4 月 5 日

目录

1 任务概览 **4**

2 代码入口与运行方式 **4**

2.1 训练入口 4

2.2 命令行参数与默认配置 4

2.3 训练框架与运行模式 5

3 模型总体架构 **6**

3.1 总体思路 6

3.2 总体调用关系 6

3.3 总体张量流摘要 6

3.4 核心模块总览 7

4 核心模块分析 **8**

4.1 WarmStartFiller：插补前双向暖启动 8

4.1.1 设计思路 8

4.1.2 功能说明 8

4.1.3 输入张量 8

4.1.4 输出张量 9

4.1.5 关键参数与状态 9

4.2 GraphLearner：按窗口动态学习变量图 9

4.2.1 设计思路 9

4.2.2 功能说明 9

4.2.3 输入张量 9

4.2.4 输出张量 10

4.2.5 关键参数与状态 10

4.3 GraphGCNReconstructor：基于动态图的跨变量重构专家 10

4.3.1 设计思路 10

4.3.2 功能说明 10

4.3.3 输入张量 10

4.3.4 输出张量 11

4.3.5 关键参数与状态 11
4.4 FrequencyOnlyReconstructor：频域重构专家 11
4.4.1 设计思路 11
4.4.2 功能说明 11
4.4.3 输入张量 11
4.4.4 输出张量 12
4.4.5 关键参数与状态 12
4.5 TwoExpertGatedImputer：双专家门控融合插补器 12
4.5.1 设计思路 12
4.5.2 功能说明 12
4.5.3 输入张量 12
4.5.4 输出张量 13
4.5.5 关键参数与状态 13
4.6 SamplingAwareTransformerAD：采样率感知异常检测器 13
4.6.1 设计思路 13
4.6.2 功能说明 13
4.6.3 输入张量 14
4.6.4 输出张量 14
4.6.5 关键参数与状态 14
4.7 FusionAnomalyModel 与损失聚合 14
4.7.1 设计思路 14
4.7.2 功能说明 15
4.7.3 输入张量 15
4.7.4 输出张量 15
4.7.5 关键参数与状态 15

5 数据处理方法 16
5.1 数据源与样本组织 16
5.2 标准化与缺失值处理 16
5.3 窗口构造 16
5.4 训练时的自监督目标构造 17
5.5 检测分数、后处理与指标 17

6 训练流程与参数更新 17
6.1 单次训练迭代 17
6.2 参数更新说明 18
6.3 训练辅助机制 19
6.4 关于 detector loss 对插补器的影响 19

7 端到端数据流 19
7.1 从原始样本到最终预测的顺序流程 19
7.2 阶段-输入-输出-状态变化总表 20

7.3 main() 的调用过程细化 21

8 关键结论与待验证问题 21

8.1 关键结论 21

8.2 待验证问题 22

8.3 本报告的证据边界 22

1 任务概览

本报告分析仓库中的 `mra.py` 及其直接依赖，目标是说明一个“多速率缺失补全 + 无监督异常检测”模型如何从原始 CSV 序列出发，经过窗口化、双专家插补、采样率感知 Transformer 重构，再形成异常分数与二分类预测。代码主体采用原生 PyTorch，没有额外 trainer 封装，也没有把模型拆成多个训练阶段；插补器与检测器在同一个 `FusionAnomalyModel` 中联合训练。

本次重点阅读的文件为：

- `mra.py`：训练入口、模型装配、损失、训练循环、推理与评分主链路。
- `graph_gcn_reconstruction.py`：观测值标准化器、动态图学习器、GCN 插补专家、随机种子与可视化辅助函数。
- `freq_reconstruction.py`：频域插补专家。
- `utils/methods/data_loading.py`：CSV 读取与缺失掩码构造。
- `utils/methods/windowing.py`：训练/评估窗口切分与测试标签构造。
- `utils/methods/postprocess.py`、`utils/methods/display.py`：分数平滑、阈值选择、指标与图表输出。

按默认数据与超参数，训练集与测试集各包含一个 `4100\times18` 的 CSV，18 个变量被代码显式分成 3 个采样率组，每组 6 个变量。基于缺失率推断得到的步长分别为 1、3、5，对应的 `rate_id` 为 3 个离散组。**证据：** `data/train/train_1.csv`、`data/test/test_C5_1.csv`、`mra.py:139-150`，以及基于代码的轻量级 shape 验证结果。

报告中的记号约定如下：B 表示 batch 大小，T 表示窗口长度（默认 50），N 表示变量数（本项目默认 18），R 表示采样率类别数（本项目默认 3），d_{model} 表示检测器 token 维度（默认 128）。

2 代码入口与运行方式

2.1 训练入口

训练入口是 `mra.py` 的 `main()`。脚本执行顺序非常直接：

1. `parse_args()` 解析命令行参数。
2. 检查 `ewaf_alpha` 和 `min_anomaly_duration` 的合法性。
3. 调用 `seed_everything()` 固定随机性，并调用 `configure_chinese_font()` 配置中文绘图字体。
4. 读取训练/测试 CSV，构造缺失掩码，做基于观测值的标准化。
5. 构造训练插补窗口、训练评分窗口和测试评分窗口。
6. 基于训练缺失率推断多速率元数据 `rate_id` 与 `stride`。
7. 实例化 `FusionAnomalyModel` 并调用 `train_model()` 联合训练。
8. 用训练好的模型重建全序列，保存补全结果、门控权重和邻接矩阵热图。
9. 重新收集训练/测试窗口统计量，组合成最终异常分数，平滑、阈值化、最短持续时间过滤后输出预测结果与图表。

   证据： `mra.py:56-125`、`mra.py:831-1098`。

2.2 命令行参数与默认配置

参数	默认值	作用
`--train-glob`	`data/train/train/train_1.csv`	训练集 CSV 路径或 glob。
`--test-glob`	`data/test/test/test_C5_1.csv`	测试集 CSV 路径或 glob。
`--seq-len`	50	全部窗口与频域专家的时序长度。
`--stride`	1	窗口滑动步长。
`--epochs`	12	训练轮数。
`--batch-size`	64	DataLoader batch 大小。
`--lr`	1e-3	AdamW 学习率。
`--weight-decay`	1e-4	AdamW 权重衰减，对全部可训练参数统一生效。
`--holdout-ratio`	0.15	每个训练窗口内，从原本可观测位置额外随机遮挡的比例。
`--graph-hidden-dim`	32	图学习器隐藏维度。
`--gcn-hidden-dim`	32	GCN 中间通道数。
`--gate-hidden-dim`	64	门控 MLP 隐藏维度。
`--rate-embed-dim`	8	插补门控使用的采样率 embedding 维度。
`--detector-d-model`	128	Transformer token 维度。
`--detector-heads`	4	多头注意力头数。
`--detector-layers`	3	编码器层数。
`--diag-target`	0.25	邻接矩阵对角权重的正则目标。
`--score-disagreement-weight`	0.25	专家分歧分数在最终异常分数中的权重。
`--score-shift-weight`	1.0	分布偏移分数在最终异常分数中的权重。
`--threshold-std-factor`	2.0	高斯阈值部分的标准差系数。
`--threshold-quantile`	0.95	训练分数分位数阈值。
`--ewaf-alpha`	0.3	指数加权异常分数平滑系数。
`--min-anomaly-duration`	50	异常段最短持续窗口长度。

证据： `mra.py:56-125`。

2.3 训练框架与运行模式

本脚本使用原生 PyTorch：

- 数据层是 `TensorDataset + DataLoader`。
- 训练循环是手写的 `for epoch` / `for batch`。
- 优化器是单个 `optim.AdamW(model.parameters(), ...)`。
- 没有学习率调度器，没有 AMP，没有梯度累积，没有 DDP 或多卡逻辑。

- 设备由 `choose_device()` 自动选择：若命令行未指定，就优先 `cuda`，否则回退到 `cpu`。

因此，从代码语义看，它是单进程、单设备、端到端联合训练的无监督检测脚本。**证据：** `mra.py:128-131`、`mra.py:548-621`。

3 模型总体架构

3.1 总体思路

模型可以概括为两层结构：

1. 第一层是 **双专家门控插补器**。它把多速率缺失序列先做双向 warm start，再并行送入图结构专家和频域专家，最后依据变量采样率和当前局部状态做逐时刻、逐变量的二专家加权融合。
2. 第二层是 **采样率感知 Transformer 检测器**。它把补全后的窗口连同观测掩码、变量 ID、采样率 ID、相位信息一起编码成时间 token，学习“正常窗口的自重构规律”，再用重构误差构成异常分数的主成分。

这不是“先单独训练插补器、再冻结后训练检测器”的两阶段流程。实际代码是在一次 forward 中同时生成插补结果与检测结果，并用总损失联合回传。**证据：** `mra.py:457-545`、`mra.py:548-621`。

3.2 总体调用关系

阶段	主调用关系
数据读取	CSV → 缺失掩码 → 标准化 → 窗口化
元数据构造	缺失率 → `rate_id`, `stride`
插补前处理	`x_true` + 结构性缺失掩码 → 随机 holdout → `x_input`
插补器	`x_input` → `WarmStartFiller` → GCN 专家 / 频域专家 → 门控融合 → `x_complete`
检测器	`x_complete` + `observed_mask` + `rate_id` + `stride` → Transformer → `detector_reconstruction`
训练损失	插补监督损失 + 图正则 + 检测重构损失 → 总损失 → backward
推理评分	检测分数 + 专家分歧 + 分布偏移 → EWAF 平滑 → 阈值化 → 时长过滤

3.3 总体张量流摘要

张量/数组名	Shape	DType	语义
`train_raw`, `test_raw`	$(4100, 18)$	`float32`	原始 CSV 数值，缺失位置初始为 NaN。
`train_mask`, `test_mask`	$(4100, 18)$	`float32`	缺失掩码，1 表示原始 CSV 缺失。
`train_scaled`, `test_scaled`	$(4100, 18)$	`float32`	基于训练集观测值标准化后的序列，缺失位被均值填充，因此变成 0。
`train_impute_windows`	$(4100, 50, 18)$	`float32`	用于联合训练和全序列补全的前填充窗口。
`train_eval_windows`	$(4001, 50, 18)$	`float32`	用于训练分数统计的标准窗口。
`test_eval_windows`	$(4000, 50, 18)$	`float32`	用于测试分数统计的标准窗口。

张量/数组名	Shape	DType	语义
`rate_id`	$(18,)$	`int64`	变量所属采样率组，实际为 $[0\times 6, 1\times 6, 2\times 6]$。
`stride`	$(18,)$	`int64`	由观测比例推断的采样步长，实际为 $[1\times 6, 3\times 6, 5\times 6]$。
`x_true`	$(B, 50, 18)$	`float32`	一个 batch 的真实训练窗口。
`structural_missing_mask`	$(B, 50, 18)$	`float32`	数据原生缺失掩码。
`holdout_mask`	$(B, 50, 18)$	`float32`	训练时额外随机遮挡的观测位置。
`x_input`	$(B, 50, 18)$	`float32`	对结构性缺失与 holdout 统一置零后的输入。
`x_seed`, `x_gcn`, `x_freq`, `x_imputed`, `x_complete`	$(B, 50, 18)$	`float32`	插补阶段的中间输出与最终补全结果。
`gate_weights`	$(B, 50, 18, 2)$	`float32`	每个时间步、每个变量对 GCN 与频域专家的归一化权重。
`adjacency`	$(B, 18, 18)$	`float32`	按窗口动态学习的变量图邻接矩阵。
`detector_reconstruction`	$(B, 50, 18)$	`float32`	检测器重构结果。
`detector_error`	$(B, 50, 18)$	`float32`	检测器绝对误差，仅用于统计与输出，不直接进入损失。
`total_loss` 等	标量	`float32`	训练时的五个损失分量及加权和。

证据： `mra.py:139-150`、`mra.py:270-509`、`mra.py:512-545`、`mra.py:847-877`，以及小批量 forward 验证结果。

3.4 核心模块总览

模块	文件/符号	核心职责	关键输入与输出
WarmStartFiller	`mra.py` / `WarmStartFiller`	在学习型插补前做双向时序填充，给专家提供非零初值。	输入 (B, T, N) 的序列与掩码，输出 warm start 序列。
图学习 + GCN 专家	`graph_gcn_reconstruction.py` / `GraphLearner`, `GraphGCNReconstructor`	从窗口统计量学习变量图，并据此做跨变量重构。	输入 warm start 与缺失掩码，输出图邻接矩阵与 GCN 重构。
频域专家	`freq_reconstruction.py` / `FrequencyOnlyReconstructor`	在频域上增强周期结构并回到时域重建。	输入 warm start，输出频域重构序列。

模块	文件/符号	核心职责	关键输入与输出
门控融合插补器	mra.py / TwoExpertGatedImputer	综合采样率 embedding 与专家差异，逐点融合两路专家结果。	输入 x_input, model_missing_mask, rate_id，输出 x_complete 等字典。
采样率感知检测器	mra.py / SamplingAwareTransformerAD	将补全窗口编码为时间 token，用自重构误差建模正常模式。	输入 x_complete, observed_mask, rate_id, stride，输出重构序列。
联合封装与损失	mra.py / FusionAnomalyModel, compute_losses	把插补器和检测器串联，并聚合五个损失分量。	输入 batch 及掩码，输出多分支结果和总损失。

4 核心模块分析

4.1 WarmStartFiller：插补前双向暖启动

4.1.1 设计思路

若把所有缺失位置直接置零送入学习模块，图结构和频域分支都要同时面对“真实 0”和“人为置零”的歧义。这里先用一个不含参数的双向填充器构造 x_seed：正向扫描得到前向最近观测值，反向扫描得到后向最近观测值，两侧都存在时取均值，只存在一侧时取该侧值，两侧都没有时保留 0。这样做的目的不是最终插补，而是给后续专家一个更平滑、更接近真实序列的初始条件。

4.1.2 功能说明

_directional_fill() 会沿时间维遍历窗口，维护“最近一次观测到的值”和“是否已经出现过观测值”的布尔状态；fill() 再把前向与后向结果合并。整个模块没有可训练参数，也不会引入新的 mask。

4.1.3 输入张量

张量名	Shape	DType	语义与来源
values	(B, T, N)	float32	训练或推理时送入插补器的 x_input，缺失位已被置零。
missing_mask	(B, T, N)	float32 / bool	模型当前视角下的缺失掩码，训练时含结构性缺失与 holdout，推理时只含结构性缺失。

4.1.4 输出张量

张量名	Shape	DType	语义与去向
`x_seed`	(B, T, N)	`float32`	双向填充后的 warm start 序列，送往 GCN 专家和频域专家。

4.1.5 关键参数与状态

- 无可训练参数，无 buffer。
- 中间状态包括 `last_value`、`has_value`、`forward_available` 和 `backward_available`。
  证据： `mra.py:185-248`。

4.2 GraphLearner：按窗口动态学习变量图

4.2.1 设计思路

图分支不是使用固定先验图，而是先从当前窗口中为每个变量抽取一个低维动态上下文，再和静态可学习节点 embedding 拼接成节点表示，进而学习边权。这个设计使邻接矩阵既受长期静态关系影响，也能随窗口内统计特征而变化，适合多变量工业序列中“当前工况下的相关结构”建模。

4.2.2 功能说明

对每个变量，`GraphLearner` 构造 4 维动态特征：

1. 观测均值；
2. 观测标准差；
3. 最近一次观测值；
4. 当前窗口内缺失比例。

然后把这 4 维动态特征与 8 维静态上下文拼成 12 维节点特征，经 MLP 编码到 32 维节点表示。边表示由 $(h_i, h_j, |h_i - h_j|, h_i \odot h_j)$ 拼接得到，再通过 `edge_mlp` 输出一条边的 logit。同时又引入两种额外关系：

- 基于可学习节点坐标的距离先验 `prior`；
- 基于归一化节点表示的余弦相似度 `cosine_similarity`。

最终，边 logit、距离先验和相似度按正系数加权相加，对角线再额外注入可学习自环强度，经 softmax、对称化和行归一化得到邻接矩阵。

4.2.3 输入张量

张量名	Shape	DType	语义与来源
`x`	(B, T, N)	`float32`	Warm start 后的窗口序列。
`mask`	(B, T, N)	`float32`	缺失掩码，1 表示当前模型视角下不可观测。

4.2.4 输出张量

张量名	Shape	DType	语义与去向
`node_dynamic`	$(B, N, 4)$	`float32`	每个变量在当前窗口内的动态统计量，中间量。
`node_repr`	$(B, N, 32)$	`float32`	节点编码，中间量。
`adj`	(B, N, N)	`float32`	对称且行归一化的动态图，送往 3 层 GCN。

4.2.5 关键参数与状态

- 标量参数：`self_loop_logit`、`prior_strength`、`similarity_strength`、`temperature`。
- 节点参数：`static_context` $\in \mathbb{R}^{N\times 8}$、`node_coords` $\in \mathbb{R}^{N\times 8}$。
- MLP 参数：`node_encoder`（$12 \to 32 \to 32$）和 `edge_mlp`（$128 \to 32 \to 1$）。
- 无冻结逻辑，上述参数在训练中全部可更新。

  证据：`graph_gcn_reconstruction.py:241-324`。

4.3 GraphGCNReconstructor：基于动态图的跨变量重构专家

4.3.1 设计思路

GCN 分支的假设是：即便某个变量当前时刻缺值，它仍可由同一时间步其他变量的值，通过变量间关系图进行消息传播得到估计。与很多时空图模型不同，这里没有再学习时序卷积，而是先用整窗统计学到一个固定于该窗口的邻接矩阵，然后对窗口内每个时间步共享这张图。

4.3.2 功能说明

`GCNLayer` 先对每个节点特征做线性变换，再执行

$$\mathrm{out}_{b,s,n,:} = \sum_{m=1}^{N} \mathrm{adj}_{b,n,m} \cdot \mathrm{x}_{b,s,m,:},$$

因此它沿变量维进行图传播，同时保留 batch 维和时间维。整个重构器由三层 GCN 组成：

1. `gcn_in`: $1 \to 32$；
2. `gcn_mid`: $32 \to 32$；
3. `gcn_out`: $32 \to 1$。

   前两层后接 ReLU、LayerNorm 和 Dropout，最后一层回到标量序列，得到 `x_gcn`。

4.3.3 输入张量

张量名	Shape	DType	语义与来源
`x`	(B, T, N)	`float32`	Warm start 序列，由 `WarmStartFiller.fill()` 产生。
`mask`	(B, T, N)	`float32`	当前输入缺失掩码，传给 `GraphLearner` 以构造动态统计量。

4.3.4 输出张量

张量名	Shape	DType	语义与去向
`x_rec` / `x_gcn`	(B, T, N)	`float32`	GCN 专家对全部变量、全部时间步的重构结果，进入门控融合与独立重构损失。
`adj`	(B, N, N)	`float32`	当前窗口动态图，同时进入图正则。

4.3.5 关键参数与状态

- 子模块 `graph`：即上节的 `GraphLearner`。
- 三层线性图卷积：`gcn_in.linear.*`、`gcn_mid.linear.*`、`gcn_out.linear.*`。
- 归一化层：`norm_in`、`norm_mid`。
- Dropout 概率固定为 0.1。

  证据：`graph_gcn_reconstruction.py:329-362`。

4.4 FrequencyOnlyReconstructor：频域重构专家

4.4.1 设计思路

频域专家假设多速率过程变量可能具有周期性、谐波或慢变频谱结构，因此先沿时间维做实数 FFT，把时域窗口映射到频域，再让网络直接学习如何增强频谱实部和虚部，最后逆变换回时域。相比图分支，它更强调单变量的时序频谱规律，而不是跨变量拓扑关系。

4.4.2 功能说明

`FrequencyImputer.forward()` 的关键步骤是：

1. 将输入从 (B, T, N) 置换到 (B, N, T)，以便按时间维做 `rfft`。
2. 对于默认 $T = 50$，频域长度 `freq_len` 为 26。
3. 把频谱实部和虚部拼成长度 52 的特征，送入两套 MLP：
   - `attention`: $52 \to 128 \to 26$，输出每个频点的门控权重；
   - `freq_enhance`: $52 \to 128 \to 52$，输出频谱增量。
4. 用注意力权重调制增量后叠加回原频谱，再用 `irfft` 回到时域。

`FrequencyOnlyReconstructor` 在此基础上再加一个共享前馈头 `output_head`（$N \to N \to N$），并以残差形式输出

$$x_{freq}^{out} = x_{freq} + \text{output_head}(x_{freq}).$$

4.4.3 输入张量

张量名	Shape	DType	语义与来源
`x`	(B, T, N)	`float32`	Warm start 序列。

4.4.4 输出张量

张量名	Shape	DType	语义与去向
`x_freq`	(B, T, N)	`float32`	频域专家的时域重构结果，进入门控融合与独立重构损失。

4.4.5 关键参数与状态

- 频域注意力 MLP：`attention.0/2`。
- 频域增强 MLP：`freq_enhance.0/2`。
- 输出头：`output_head.0/2`。
- 不显式依赖掩码；它完全以 warm start 后的完整数值窗口为输入。
  证据：`freq_reconstruction.py:95-141`。

4.5 TwoExpertGatedImputer：双专家门控融合插补器

4.5.1 设计思路

双专家结构的核心假设是：不同变量、不同时间步适合依赖的插补机制不同。图分支擅长利用同一时刻变量间相关性，频域分支擅长利用单变量的时序频谱模式；因此代码没有简单平均两者，而是为每个变量、每个时间步动态学习一个二元门控。

4.5.2 功能说明

该模块按以下顺序工作：

1. 用 `WarmStartFiller.fill()` 得到 `x_seed`。
2. 并行计算 `x_gcn` 和 `x_freq`。
3. 将 `rate_id` 送入 `nn.Embedding(R, 8)`，扩展到 $(B, T, N, 8)$。
4. 构造门控特征
$$[x_{seed},\ mask,\ x_{gcn},\ x_{freq},\ x_{gcn} - x_{seed},\ x_{freq} - x_{seed}],$$
   这 6 个标量特征与 8 维采样率 embedding 拼成 14 维输入。
5. 经门控 MLP（$14 \to 64 \to 64 \to 2$）输出两位专家 logit，softmax 后得到 `gate_weights`。
6. 对堆叠后的专家结果 `experts = stack([x_gcn, x_freq])` 做加权求和，得到 `x_imputed`。
7. 仅在当前模型认定为缺失的位置，用 `x_imputed` 替换 `x_input`，形成 `x_complete`。

需要注意，门控是在所有位置都计算的，但 `x_complete` 只在 `model_missing_mask==1` 处真正采用融合结果；对原本可观测且未 holdout 的位置，`x_complete` 直接保留 `x_input`。

4.5.3 输入张量

张量名	Shape	DType	语义与来源
`x_input`	(B, T, N)	`float32`	对结构性缺失和 holdout 置零后的模型输入。
`model_missing_mask`	(B, T, N)	`float32`	模型视角的缺失掩码，训练时等于结构性缺失与 holdout 的并集。
`rate_id`	$(N,)$	`int64`	每个变量的采样率类别 ID。

4.5.4 输出张量

张量名	Shape	DType	语义与去向
`x_seed`	(B, T, N)	`float32`	warm start 序列。
`x_gcn`	(B, T, N)	`float32`	GCN 专家输出。
`x_freq`	(B, T, N)	`float32`	频域专家输出。
`x_imputed`	(B, T, N)	`float32`	两专家加权融合的纯插补结果。
`x_complete`	(B, T, N)	`float32`	对缺失位替换后的完整窗口，送往检测器。
`gate_weights`	$(B, T, N, 2)$	`float32`	每个变量在每个时间步对 GCN/Freq 两专家的权重。
`adjacency`	(B, N, N)	`float32`	由 GCN 分支学习的变量图。

4.5.5 关键参数与状态

- 图专家参数：`imputer.gcn.*`。
- 频域专家参数：`imputer.freq.*`。
- 采样率 embedding：`imputer.rate_embedding.weight` $\in \mathbb{R}^{R\times 8}$。
- 门控 MLP 参数：`imputer.gate.0/2/4.*`。

  证据： `mra.py:270-340`。

4.6 SamplingAwareTransformerAD：采样率感知异常检测器

4.6.1 设计思路

检测器不直接在原始缺失序列上工作，而是吃 `x_complete`，因此它学习的是“补全后的正常窗口应该长什么样”。多速率信息并不是只通过一个全局 rate embedding 喂进去，而是拆成三部分：

- 变量身份 embedding；
- 采样率类别 embedding；
- 根据步长构造的相位 embedding。

这样做的含义是：对于采样步长较大的变量，模型能显式知道某个时间步处在它采样周期中的哪个相位位置，而不是把不规则采样完全隐含到数值本身。

4.6.2 功能说明

前向过程可拆成 5 步：

1. 为 N 个变量生成固定的变量 embedding，为 N 个变量的 `rate_id` 生成采样率 embedding。
2. 根据
$$\text{phase}_{t,n} = \frac{\text{remainder}(t, \text{stride}_n)}{\text{stride}_n}$$
生成 $(T, N, 1)$ 的相位标量，再经 `phase_mlp` 得到 8 维相位向量。
3. 对每个变量的每个时间步拼接
$$[x_{complete},\ observed_mask,\ variable_embed,\ rate_embed,\ phase_embed]$$
共 26 维，送入 `per_variable_proj` 得到 16 维变量级特征。

4. 把整窗输入展平为时间 token：原始值 18 维 + 观测掩码 18 维 + 展平后的变量级特征 $18 \times 16 = 288$ 维，总计 324 维，再投影到 `d_model=128`。
5. 添加正弦位置编码，经 3 层 `TransformerEncoder` 后，用重构头输出每个时间步的增量 `delta`，最终返回 `x_complete + delta`。

这里的 attention 作用在时间 token 之间，而不是变量之间；变量关系在进入 Transformer 前已经被编码进展平后的 token 特征中。

4.6.3 输入张量

张量名	Shape	DType	语义与来源
`x_complete`	(B, T, N)	`float32`	插补后的完整窗口。
`observed_mask`	(B, T, N)	`float32`	结构性观测掩码，等于 $1 - \text{structural_missing_mask}$。
`rate_id`	$(N,)$	`int64`	采样率类别 ID。
`stride`	$(N,)$	`int64`	每个变量的推断采样步长。

4.6.4 输出张量

张量名	Shape	DType	语义与去向
`detector_reconstruction`	(B, T, N)	`float32`	检测器的重构结果，用于 detector loss 与检测分数。
`tokens`	$(B, T, 128)$	`float32`	时间 token，中间表示。
`encoded`	$(B, T, 128)$	`float32`	Transformer 编码结果，中间表示。

4.6.5 关键参数与状态

- 变量 embedding：`variable_embedding.weight` $\in \mathbb{R}^{18\times 8}$。
- 采样率 embedding：`rate_embedding.weight` $\in \mathbb{R}^{3\times 8}$。
- 相位 MLP：`phase_mlp.0/2.*`。
- 变量级投影：`per_variable_proj.0/2.*`，默认输入 26 维、输出 16 维。
- token 投影：`token_proj.*`，默认 $324 \to 128$。
- Transformer 编码器：3 层 `encoder.layers.*`。
- 重构头：`reconstruction_head.*`。
- 非可训练 buffer：正弦位置编码 `pe`，由 `register_buffer(..., persistent=False)` 保存。

  证据： `mra.py:251-267`、`mra.py:343-455`。

4.7 FusionAnomalyModel 与损失聚合

4.7.1 设计思路

`FusionAnomalyModel` 只是一个薄封装，但它决定了两个关键事实：

- 插补器和检测器在同一次 forward 中串联，而不是分阶段运行。
- 检测器吃的是 `x_complete`，因此 detector loss 不仅更新检测器，还可能沿着输入路径间接影响插补器。

4.7.2 功能说明

forward 时先调用 `self.imputer(...)` 得到插补字典，再把 `structural_missing_mask` 转成 `observed_mask`，送入检测器得到 `detector_reconstruction` 和绝对误差 `detector_error`。随后 `compute_losses()` 聚合 5 个损失：

$$
\begin{aligned}
L_{\text{fusion}} &= \text{MSE}_{holdout}(x_{imputed}, x_{true}) \\
L_{\text{gcn}} &= \text{MSE}_{holdout}(x_{gcn}, x_{true}) \\
L_{\text{freq}} &= \text{MSE}_{holdout}(x_{freq}, x_{true}) \\
L_{\text{graph}} &= \text{mean}\left((\text{diag}(A) - \text{diag_target})^2\right) \\
L_{\text{det}} &= \text{MSE}_{observed}(\hat{x}_{det}, \text{stopgrad}(x_{complete}))
\end{aligned}
$$

总损失为

$$
L = 1.0L_{\text{fusion}} + 0.4L_{\text{gcn}} + 0.4L_{\text{freq}} + 0.5L_{\text{det}} + 0.1L_{\text{graph}}.
$$

其中一个关键实现细节是 `detector_target = outputs["x_complete"].detach()`：目标端的 `x_complete` 被截断梯度，但检测器输出依然依赖 `x_complete` 作为输入，所以 `L_det` 对插补器仍保留“经检测器输入路径回传”的间接梯度。

4.7.3 输入张量

张量名	Shape	DType	语义与来源
`x_input`	(B, T, N)	`float32`	训练或推理窗口的模型输入。
`model_missing_mask`	(B, T, N)	`float32`	插补器视角的缺失掩码。
`structural_missing_mask`	(B, T, N)	`float32`	数据原生缺失掩码。
`x_true`	(B, T, N)	`float32`	真实训练窗口，仅在训练损失中出现。
`holdout_mask`	(B, T, N)	`float32`	额外随机遮挡位，用于构造自监督插补目标。

4.7.4 输出张量

张量名	Shape	DType	语义与去向
`outputs`	字典	–	同时包含插补器与检测器输出。
`fusion_loss` 等 5 个分量	标量	`float32`	分别约束门控插补、单专家插补、图结构和检测器重构。
`total_loss`	标量	`float32`	反向传播入口。

4.7.5 关键参数与状态

- 封装层本身无新参数；所有可训练参数来自 `imputer` 和 `detector`。
- `LossWeights` 只是 dataclass 配置，不是参数，也不会参与优化。

  证据： `mra.py:457-545`。

5 数据处理方法

5.1 数据源与样本组织

数据读取由 `load_csv_glob_with_mask()` 完成。该函数对 glob 匹配到的所有 CSV 逐个执行：

1. 用 `pandas.read_csv(..., header=None)` 读成二维表；
2. 转为 `float32` 数组；
3. 以 `np.isnan(arr)` 生成缺失掩码；
4. 检查列数一致后沿时间维拼接。

因此样本的基本组织单位不是“独立文件 = 一个样本”，而是“所有 CSV 按时间顺序拼成一条长序列，再切成滑动窗口”。本项目默认训练集和测试集各只有一个文件，所以最终得到两条长度 4100、变量数 18 的多变量时间序列。**证据：**`utils/methods/data_loading.py:10-98`。

5.2 标准化与缺失值处理

`ObservedStandardScaler` 只利用 **观测到的值**统计每个变量的均值和标准差：

- `fit()` 时跳过缺失位与非有限值。
- `transform()` 时先把缺失位用该变量均值填充，再做 z-score 标准化。
- 因为填充值正好是均值，所以标准化后缺失位变成 0。

这一设计与后续 `x_input = x_true.masked_fill(mask, 0)` 形成了统一语义：模型看到的缺失位数值始终是 0，但掩码单独保留了“这个 0 是缺失位还是正常值”的上下文信息。**证据：**`graph_gcn_reconstruction.py:167-206`、`mra.py:584-593`。

5.3 窗口构造

方法/组件	所在位置	触发阶段	作用与影响
`build_windows`	`utils/methods/windowing.py:79-106`	训练插补、全序列重建	使用前填充窗口。对前 $T-1$ 个时间步，通过重复首行构造满长度窗口，因此输出窗口数与原始序列长度一致。
`build_standard_windows`	`utils/methods/windowing.py:109-130`	训练评分统计	不做前填充，从 `start_index=99` 开始生成标准长度窗口；默认产生 4001 个窗口。
`build_prompt_test_windows`	`utils/methods/windowing.py:133-167`	测试评分统计	同样生成标准窗口，但默认只取 4000 个，并把前 2000 个标为 0、后 2000 个标为 1。标签来自窗口位置，不来自 CSV 文件内容。
`TensorDataset` + `DataLoader`	`mra.py:563-567`、`mra.py:634-638`、`mra.py:675-679`	训练与评估 batch 化	无自定义 `Dataset` 或 `collate_fn`；窗口和掩码直接按 batch 堆叠。

这里有两个非常关键的实现细节：

- **训练窗口**用前填充版本，是为了让前几个时间步也能产生对应的最后一步重建结果。
- **测试标签**是通过 `split_index=2000` 人工构造的，即假设测试窗口前半段正常、后半段异常。模型并没有从 CSV 中读取真实 fault 标签。

证据：`utils/methods/windowing.py:6-10`、`utils/methods/windowing.py:73-76`、`mra.py:854-877`。

5.4 训练时的自监督目标构造

训练没有人工异常标签，插补监督也不是直接用原生缺失位，因为原生缺失位没有真值。代码采用“观测值再遮挡”策略：

1. 在 `sample_holdout_mask()` 中，从原本可观测位置随机抽取约 15% 作为 holdout。
2. 若某个样本至少有观测值但随机采样结果为空，则强制再抽一个位置，确保每个可训练窗口至少有一个监督点。
3. 用 `model_missing_mask = max(structural_missing_mask, holdout_mask)` 把结构性缺失与自监督缺失统一起来。
4. 仅在 `holdout_mask` 位置计算插补损失。

这样一来，图专家、频域专家和门控融合器都能在有真值的位置接受监督，而不需要知道结构性缺失的真值。**证据：**`mra.py:162-182`、`mra.py:584-601`。

5.5 检测分数、后处理与指标

模型最终并非只用 detector loss 本身作为异常分数，而是把三类统计量融合：

1. **detector_score**：检测器重构误差在观测位上的平均平方误差。
2. **disagreement_score**：图专家和频域专家输出的平均绝对差。
3. **shift_score**：对补全后窗口提取 108 维统计特征后，相对训练分布的标准化偏移强度。

`build_window_feature_matrix()` 假定 18 个特征按 $[0:6],[6:12],[12:18]$ 三个组排列，并对每组以及全集分别取时间均值和标准差，再补充平均绝对一阶差分和前 5 个非零频点 FFT 幅值均值，总计 108 维特征。之后：

- 用训练集均值和标准差分别标准化三类分数；
- 最终异常分数为
$$|\text{detector_z}| + 0.25 \cdot \text{gap_z} + 1.0 \cdot \text{shift_z};$$
- 对训练分数和测试分数做 EWAF 平滑，其中测试集按 $[2000, 2000]$ 两段分别平滑，避免越过正常/异常边界传播；
- 阈值取 $\max(\mu + 2\sigma,\ q_{0.95})$；
- 最后再用 `apply_min_anomaly_duration()` 删除长度不足 50 的短异常片段。

证据：`mra.py:665-829`、`mra.py:967-1034`、`utils/methods/postprocess.py:6-99`。

6 训练流程与参数更新

6.1 单次训练迭代

一次训练 iteration 的真实执行顺序如下：

1. DataLoader 从训练窗口张量中取出两个字段：`x_true` 和 `structural_missing_mask`。

2. 将二者搬到目标设备。
3. 调用 `sample_holdout_mask()` 构造额外监督掩码 `holdout_mask`。
4. 通过 `model_missing_mask = max(structural, holdout)` 合并缺失位。
5. 用合并掩码把 `x_true` 中对应位置置零，得到 `x_input`。
6. 调用 `model(...)`：
   - 插补器输出 `x_seed`、`x_gcn`、`x_freq`、`x_imputed`、`x_complete`、`gate_weights`、`adjacency`；
   - 检测器输出 `detector_reconstruction` 与 `detector_error`。
7. 调用 `compute_losses()` 计算 5 个损失及总损失。
8. `optimizer.zero_grad()` 清梯度。
9. `losses["total_loss"].backward()` 触发反向传播。
10. `clip_grad_norm_(model.parameters(), 1.0)` 做全模型梯度裁剪。
11. `optimizer.step()` 用 AdamW 更新参数。
12. 把当前 batch 的各项损失累计到 epoch 统计量。

    这里没有 `scheduler.step()`，也没有 `scaler.step()` 或混合精度逻辑。**证据：** `mra.py:548-621`。

6.2 参数更新说明

参数组	更新时机	更新规则	说明
图学习器参数	每个 step	AdamW	包括 `self_loop_logit`、`prior_strength`、`similarity_strength`、`temperature`、`static_context`、`node_coords`、`node_encoder.*`、`edge_mlp.*`。直接受 `gcn_loss`、`graph_loss` 影响，也通过 `fusion_loss` 和 detector 输入路径间接受影响。
GCN 重构器参数	每个 step	AdamW	包括 `gcn_in/mid/out.linear.*` 与 `norm_in/mid.*`。直接受 `gcn_loss`、`fusion_loss` 影响，并可能通过 detector 输入路径间接受 `detector_loss` 影响。
频域专家参数	每个 step	AdamW	包括 `attention.*`、`freq_enhance.*`、`output_head.*`。直接受 `freq_loss`、`fusion_loss` 影响，并可能通过 detector 输入路径间接受 `detector_loss` 影响。
门控与采样率 embedding	每个 step	AdamW	包括 `rate_embedding.weight` 与 `gate.*`。不受 `gcn_loss` 或 `freq_loss` 直接监督，但受 `fusion_loss` 直接约束，也会通过 `x_complete` 输入路径接收 detector loss 的梯度。

参数组	更新时机	更新规则	说明
检测器参数	每个 step	AdamW	包括变量 embedding、采样率 embedding、相位 MLP、变量级投影、token 投影、3 层 Transformer 编码器和重构头。它们只通过 `detector_loss` 被直接训练。
位置编码 buffer	从不更新	非参数	`position_encoding.pe` 是 buffer，不出现在 `model.parameters()`，不会被优化器更新。

6.3 训练辅助机制

- **优化器**：`optim.AdamW(model.parameters(), lr, weight_decay)`，所有参数共享同一个学习率和权重衰减，没有 param group。
- **梯度裁剪**：全模型 L2 范数裁剪到 1.0。
- **随机性控制**：`seed_everything()` 固定 Python、NumPy、PyTorch 与 cuDNN 行为。
- **未使用的机制**：没有 AMP、没有 EMA、没有冻结/解冻、没有梯度累积、没有分布式训练、没有 checkpoint 恢复逻辑。

  证据： `graph_gcn_reconstruction.py:115-121`、`mra.py:568-606`。

6.4 关于 detector loss 对插补器的影响

这段代码里最容易被忽略的更新路径是：

$$L_{\text{det}} = \text{MSE}(\hat{x}_{det}(x_{complete}), \text{stopgrad}(x_{complete})).$$

虽然目标端做了 `detach`，但输入端没有。因此只要 `detector_reconstruction` 对 `x_complete` 敏感，`L_det` 就会通过

$$\hat{x}_{det} \to x_{complete} \to x_{imputed} \to \text{gate / experts}$$

这条路径把梯度回传给插补器。这个设计会鼓励插补器生成“更利于检测器建模正常模式”的补全结果，而不只是单点插补误差最小的结果。**证据：** `mra.py:500-545`。

7 端到端数据流

7.1 从原始样本到最终预测的顺序流程

1. **读文件**：`load_csv_glob_with_mask()` 把 CSV 读成 $(4100, 18)$ 数组，并依据 NaN 生成缺失掩码。
2. **标准化**：`ObservedStandardScaler.fit(train_raw, train_mask)` 用训练观测值估计均值与方差，再对训练/测试都执行 `transform()`。
3. **构造窗口**：训练插补窗口用 `build_windows()`，训练评分窗口用 `build_standard_windows()`，测试评分窗口和伪标签用 `build_prompt_test_windows()`。
4. **推断速率元数据**：`infer_rate_metadata()` 依据每列缺失比例推断采样步长和离散 rate ID。
5. **训练阶段建 batch**：DataLoader 提供 `x_true` 与 `structural_missing_mask`，代码再在线采样 `holdout_mask`。

6. **形成模型输入**：把结构性缺失与 holdout 位置统一置零，得到 `x_input`。
7. **插补前处理**：`WarmStartFiller` 用前向/后向最近观测值生成 `x_seed`。
8. **双专家重构**：GCN 专家根据整窗动态图进行跨变量传播；频域专家根据 FFT 增强单变量谱结构。
9. **门控融合**：门控网络按变量、按时间步融合两位专家结果，生成 `x_imputed`；再仅在缺失位写回，得到 `x_complete`。
10. **异常检测**：采样率感知 Transformer 把 `x_complete` 编成时间 token，并输出 `detector_reconstruction`。
11. **损失聚合**：五项损失加权求和形成 `total_loss`。
12. **反向传播**：`backward()` 将梯度传回检测器、门控、双专家和图学习器；`optimizer.step()` 更新所有可训练参数。
13. **训练后重建全序列**：`reconstruct_full_sequence()` 取每个窗口最后一步的 `x_complete`，拼成长度 4100 的补全序列。
14. **收集评分统计**：`collect_window_statistics()` 统计 detector 分数、专家分歧和门控熵；随后再计算分布偏移分数。
15. **后处理与预测**：融合分数、EWAF 平滑、阈值计算、最短异常时长过滤，最终生成 `prediction` 与各类输出文件。

7.2 阶段-输入-输出-状态变化总表

阶段	输入	输出	说明
读取	原始 CSV	`raw`, `mask`	通过 NaN 构造缺失掩码。
标准化	`train_raw`, `train_mask`	`train_scaled`, `test_scaled`	缺失位先填均值再标准化，值变为 0。
窗口化	标准化序列与掩码	三类窗口数组	训练窗口前填充；测试标签由窗口位置构造。
速率元数据	训练缺失掩码	`rate_id`, `stride`	根据观测比例反推采样步长。
批采样	窗口数组	`x_true`, `structural_missing_mask`	直接堆叠，无 `collate_fn`。
自监督遮挡	原始 batch	`holdout_mask`, `x_input`	仅从可观测点随机采样 holdout。
插补器	`x_input`, `model_missing_mask`, `rate_id`	`x_complete` 等字典	双专家输出通过门控融合。
检测器	`x_complete`, `observed_mask`, `rate_id`, `stride`	`detector_reconstruction`	时间 token 上做 3 层 Transformer 编码。

阶段	输入	输出	说明
损失	模型输出 + `x_true` + 掩码	5 项 loss + 总损失	插补监督只看 holdout，检测监督只看结构性观测位。
更新	总损失	新参数值	AdamW 更新全部参数，并做梯度裁剪。
评分后处理	训练/测试统计量	最终预测与指标	三路分数组合、EWAF、阈值、最小时长过滤。

7.3 main() 的调用过程细化

`main()` 不只是“训练模型”，还把训练后的离线分析流程一并实现了。按实际代码顺序，它还会继续：

1. 保存 `model.pt`，其中包括 `model_state_dict`、命令行参数、`rate_id`、`stride` 和 `loss_weights`。
2. 对训练集和测试集执行完整序列重建，并保存 `train\_completed.csv`、`test\_completed.csv`。
3. 汇总平均门控权重并保存成 `train\_gate\_weights.csv`、`test\_gate\_weights.csv`。
4. 保存训练邻接矩阵热图、训练曲线与异常分数曲线。
5. 输出 `metrics.json` 和 `test\_predictions.csv` 作为最终评估结果。

因此，`mra.py` 其实承担了“训练脚本 + 推理脚本 + 离线分析脚本”的三重职责。**证据：** `mra.py: 917-1094`。

8 关键结论与待验证问题

8.1 关键结论

- **确认事实**：这是一个端到端联合训练的“插补器 + 检测器”系统，而不是先补全再冻结检测的两阶段方案。
- **确认事实**：多速率信息在两个层面被利用。插补器通过 `rate_id` embedding 调整专家权重；检测器同时利用 `rate_id` 和由 `stride` 推导出的 phase 信息编码采样规律。
- **确认事实**：训练监督由两部分组成。插补监督来自观测值再遮挡的 holdout，自身不需要真实异常标签；检测监督来自补全窗口的自重构误差。
- **确认事实**：最终异常分数并非单一 detector error，而是 detector 分数、专家分歧和分布偏移三路信号的融合。
- **确认事实**：测试标签由窗口位置人工构造，前 2000 个窗口视为正常、后 2000 个窗口视为异常；这对实验设定是一个强约束。
- **推断：** 由于 `build_window_feature_matrix()` 固定使用 `0:6`、`6:12`、`12:18` 三组切片，代码隐含假设“18 个变量按三个采样率组顺序排列”。若未来变量顺序或数量变化，分布偏移特征的语义会失真。

8.2 待验证问题

- 需要结合实验设定确认：测试集“前 2000 正常、后 2000 异常”的假设是否与数据生成过程完全一致，否则 `metrics` 的解释会偏离真实标签。
- 需要结合训练日志确认：五项损失在实际训练中是否平衡，尤其是 `detector_loss` 经输入路径对插补器的间接约束强度有多大。
- 需要结合消融实验确认：图专家与频域专家各自对不同变量组的贡献是否与门控权重统计一致。
- 需要结合代码意图确认：`build_prompt_test_windows()` 默认固定为 4000 个窗口，因此不会覆盖测试序列最后一个时间点，这是否为有意设计。

8.3 本报告的证据边界

本报告基于静态代码阅读和一次小批量 shape 验证完成，没有完整跑通 12 个 epoch 的训练，因此：

- 已确认模块结构、张量接口、损失组成、参数更新路径和后处理逻辑。
- 未报告任何“真实训练中损失下降到多少”或“最终检测指标是多少”的数值，因为这些需要完整运行日志才能严谨给出。